Proyecto: Simulador de Dirección Técnica de Fútbol (*Pro Manager Studio*)

Módulo: Núcleo de Gestión, Motor de Partidos y Sistema de Vestuario

0.-Propósito y Alcance

Propósito: El presente documento tiene como propósito establecer la especificación formal, el diseño de arquitectura y la estrategia de calidad para el desarrollo del software *Pro Manager Studio* (Simulador de Dirección Técnica de Fútbol). Su objetivo es servir como guía técnica fundamental durante todo el ciclo de vida del proyecto, asegurando la trazabilidad desde la conceptual

1. Contexto General del Proyecto

***Pro Manager Studio* es una aplicación de simulación y gestión técnica/administrativa de fútbol diseñada para ofrecer una experiencia inmersiva en la que el usuario asume el rol completo de un Director Técnico (DT). El juego combina el aspecto táctico en tiempo real con la gestión humana, la economía del club y el desarrollo de un legado deportivo desde categorías inferiores.**

2. Usuarios, Visión y Escenario

Visión del Producto

- **PARA aficionados al fútbol y apasionados de los videojuegos de gestión deportiva,**
- **QUIENES buscan una experiencia inmersiva de dirección técnica que vaya más allá del aspecto táctico, abordando la gestión humana, financiera y la construcción de un legado,**
- **EL *Pro Manager Studio* ES UN videojuego de simulación y gestión deportiva,**
- **QUE permite asumir el rol integral de un DT (alineaciones, mercado de fichajes, finanzas, cantera, decisiones tácticas e interacciones disciplinarias),**
- **A DIFERENCIA DE simuladores de fútbol de corte arcade o de gestión puramente estadística,**

- **NUESTRO PRODUCTO integra un sistema de narrativa dinámica con historias propias para los jugadores, progresión orgánica y la ocurrencia de eventos imprevistos.**

Proto-Personas

1. **Carlos "El Táctico" Mendoza (24 años): Enfocado en el análisis profundo de datos, esquemas tácticos, desarrollo de cantera y progresión de ligas inferiores.**
2. **Sofía "La Directora" Ramos (19 años): Enfocada en la narrativa del vestuario, la resolución de conflictos de ego, ruedas de prensa e historias personales de los jugadores.**

Escenario Crítico de Uso

Faltan 24 horas para un partido decisivo de ascenso. El delantero estrella discute en el entrenamiento con el capitán por rumores de transferencia. El DT debe intervenir inmediatamente: sancionar al goleador enviándolo a la banca para mantener la disciplina, o mediar diplomáticamente arriesgándose a perder autoridad. Durante el partido, además de lidiar con la moral del vestuario, el portero titular se lesiona en el minuto 60, obligando al DT a realizar ajustes tácticos y sustituciones de emergencia bajo alta presión.

3. Historias de Usuario y Backlog Priorizado

- **HU-01: Configuración Táctica y Alineación Pre-Partido.**
- **HU-02: Búsqueda y Reclutamiento de Talento (Ojeadores).**
- **HU-03: Negociación de Fichajes y Transferencias.**
- **HU-04: Resolución de Conflictos de Vestuario e Historias Personales.**
- **HU-05: Toma de Decisiones Tácticas en Tiempo Real durante el Partido.**
- **HU-06: Administración Financiera y Presupuesto del Club.**

Backlog Inicial Priorizado

Prioridad	ID	Historia de Usuario	Justificación de Ingeniería
Alta	**HU-01**	**Configuración Táctica y Alineación**	**Núcleo del juego (*Core Loop*): Base para simular partidos.**
Alta	**HU-05**	**Decisiones Tácticas en Tiempo Real**	**Motor interactivo principal de la simulación del encuentro.**
Alta	**HU-04**	**Conflictos de Vestuario**	**Propuesta diferencial: gestión de personalidades y moral.**
Media	**HU-03**	**Negociación de Fichajes**	**Necesario para la progresión del club a mediano plazo.**
Media	**HU-02**	**Búsqueda de Talento y Cantera**	**Extiende la profundidad del modo carrera/legado.**
Media	**HU-06**	**Administración Financiera**	**Define las restricciones económicas del sistema.**

4. Requisitos, Calidad y Reglas de Negocio

- **Requisito Funcional: El sistema debe simular partidos en tiempo real, procesar las variables de personalidad del vestuario y permitir cambios tácticos inmediatos.**
- **Requisito de Calidad: La simulación matemática y la evaluación de eventos deben ejecutarse sin degradar el rendimiento técnico o los cuadros por segundo de la interfaz.**
- **Restricción: El motor de cálculo de dominio debe ser independiente del motor gráfico o de la interfaz de usuario.**
- **Reglas de Negocio:**
    - **Un jugador no puede tener un valor de atributo (moral, rendimiento, física) fuera del rango cerrado $[1, 100]$.**

- **Ningún club puede exceder el presupuesto de fichajes ni el límite de masa salarial libre.**
- **Un jugador en estado *Rebelde* requiere un proceso de mediación o sanción antes de volver al estado *Estable*.**

5. Caso de Uso CU-01 — Gestión de Crisis de Vestuario y Cambios en Partido

- **Nombre: Registrar Cambios Tácticos y Gestión de Crisis en Partido.**
- **Actor Principal: Director Técnico (Usuario).**
- **Objetivo: Ajustar la estrategia del equipo e intervenir ante imprevistos (lesiones, bajas morales, roces) durante el transcurso de un encuentro.**
- **Precondiciones: La simulación de un partido oficial está activa.**

Flujo Principal:

1. **El usuario solicita pausar la simulación o el sistema se detiene por una incidencia crítica.**
2. **El sistema despliega el panel de control con el estado de los jugadores y el marcador.**
3. **El usuario selecciona un jugador lesionado/agotado y realiza la sustitución.**
4. **El usuario ajusta la mentalidad táctica y emite instrucciones desde la banda.**
5. **El sistema valida las reglas de sustitución y aplica los cambios a la simulación.**
6. **El sistema confirma la aplicación de los ajustes y reanuda el encuentro.**

Excepciones:

- **3a. Jugador inhabilitado: El sistema notifica la sanción y solicita elegir otro suplente.**
- **5a. Límite de cambios alcanzado: El sistema bloquea la sustitución adicional.**

6. Modelo de Dominio

El modelo conceptual captura las entidades del problema y sus relaciones:

- **Club: Entidad institucional con atributos de *presupuesto*, *reputación* y plantilla asignada.**
- **DirectorTecnico: Representación del avatar del usuario con *experiencia* y *reputación*.**
- **Jugador: Integrante con *posición*, *atributos físicos*, *moral* y *personalidad*.**
- **Partido: Entidad donde se simula el enfrentamiento, se llevan marcadores e *incidencias*.**
- **EventoVestuario: Acontecimientos interactivos que alteran la moral del equipo.**

7. Diseño de Software: Responsabilidades y GRASP

Se aplican los patrones GRASP para justificar el diseño orientado a objetos:

- **Information Expert: La clase Jugador es la experta para calcular su rendimiento y nivel de moral, pues conoce sus propios atributos.**
- **Creator: Partido crea las instancias de IncidenciaJuego ocurridas durante la simulación.**
- **Controller: ControladorPartido y ControladorVestuario coordinan las peticiones desde la interfaz hacia el dominio.**
- **Low Coupling & High Cohesion: Separation entre el motor de simulación (SimuladorPartido), la gestión de personalidades (GestorVestuario) y las finanzas del Club.**

8. Clases de Diseño y Visibilidad

Estructura de Clases (DCD)

Plaintext

+---+

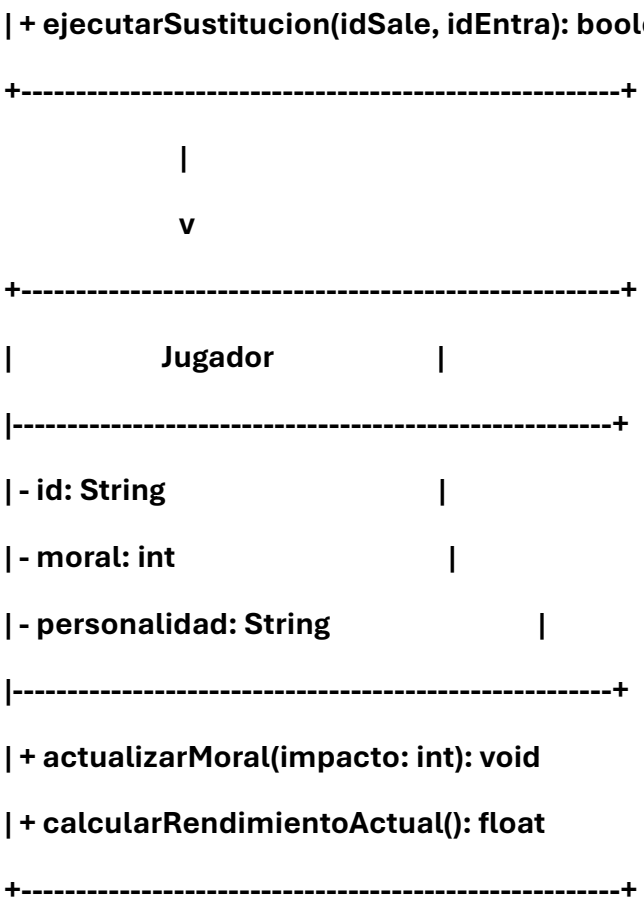
| ControladorPartido |

|---+

| - partidoActual: Partido |

|---+

| + pausarPartido(): void |



Decisiones de Visibilidad

1. **Por Parámetro: Transmisión de referencias temporales de Jugador a los controladores.**
2. **Por Atributo: El Club conserva una lista interna de sus Jugadores.**

9. Arquitectura Inicial

Organización estricta en 4 Capas:

1. **Capa de Presentación: Renderizado de interfaces, gráficos y menús.**
2. **Capa de Aplicación / Control: Coordinación de casos de uso sin lógica de negocio.**
3. **Capa de Dominio: Motor de partidos, reglas financieras y modelos de comportamiento de jugadores.**
4. **Capa de Infraestructura: Persistencia de partidas guardadas, lectura de archivos de datos e integración externa.**

10. Diseño Dinámico

Flujo de Secuencia (Gestión de Crisis)

1. **El DT solicita revisar la crisis → ControladorVestuario.**
2. **ControladorVestuario obtiene los datos del conflicto → GestorEventos.**
3. **El DT elige una acción de sanción.**
4. **Fragmento alt: Si sanciona, reduce moral en Jugador y solicita aval a Club. Si dialoga, equilibra la moral.**

Máquina de Estados de Jugador

Estable ⟶ Motivado (Elogio / Victoria)

Estable ⟶ Molesto (Falta de minutos)

Molesto ⟶ Rebelde (Sanción severa)

Rebelde ⟶ Estable (Mediación completada)

11. Integración y Trazabilidad

Problema de Gestión ⟶ HU-04 / HU-05 ⟶ CU-01 ⟶ DCD y GRASP ⟶ Diseño Dinámico ⟶ TDD y Pruebas

Cada elemento del código se deriva directamente de las especificaciones de análisis y diseño.

12. Criterios de Aceptación y Calidad del Núcleo

- **TDD (Red-Green-Refactor): Desarrollo guiado por pruebas en reglas clave.**
- **Particiones de Equivalencia y Valores Límite: Pruebas estrictas en atributos $[1, 100]$.**
- **Quality Gate Automatizado:**
  - **0 pruebas fallidas.**
  - **Cobertura de código $> 80\%$.**
  - **Revisión de código por pares (*Code Review*) obligatoria.**

13. Organización Recomendada del Proyecto

Estructura de repositorio propuesta para GitHub:

Plaintext

/

├── docs/

│ ├── TALLER_SESION_1_A_3.md

│ ├── TALLER_SESION_4.md

│ └── TALLER_SESION_6.md

├── src/

│ ├── Presentation/

│ ├── Application/

│ ├── Domain/

│ └── Infrastructure/

└── tests/

├── UnitTests/

└── IntegrationTests/

14. Próximos Pasos

1. **Crear el repositorio en GitHub e inicializar las ramas (main, develop).**
2. **Configurar las reglas de *Quality Gate* y la integración continua (CI).**
3. **Iniciar el desarrollo del primer ciclo *Vertical Slice* enfocándose en el cálculo de rendimiento de los jugadores.**

15. Cierre

El proyecto ha completado exitosamente su fase conceptual, de análisis de requisitos, diseño de arquitectura orientación a objetos, modelado dinámico y diseño de estrategia de pruebas automatizadas. La aplicación cuenta con una base sólida de ingeniería de software preparada para su fase de construcción e integración continua.